תרגיל בית 2 — האם הקליע יעבור את הקיר?

תכנות מדעי לפיזיקאים | שבוע 2 | נושא מרכזי: תנאים (`if`, `elseif`, `else`)

מטרת התרגיל

בתרגיל זה תכתבו תוכנית שלמה ב־Julia: התוכנית תקרא חמישה מספרים מהמקלדת, תבדוק שהם עומדים בדרישות המודל, תחשב את גובה הקליע במיקום הקיר ותדפיס את התוצאה. תשלימו בעצמכם את פקודות הקלט, החישובים, התנאים ופקודות הפלט. בהתאם לתוצאה, התוכנית תדפיס שורת תוצאה אחת או שלוש שורות תוצאה. אין צורך להגדיר פונקציות או לכתוב לולאות.

המודל הפיזיקלי

קליע נורה מגובה הקרקע במהירות התחלתית `speed_m_s` ובזווית `angle_deg` מעל האופק. קיר נמצא במרחק אופקי `wall_x_m`, וגובהו `wall_height_m`. גודל תאוצת הכובד הוא `gravity_m_s2`. הכיוון האופקי החיובי הוא לעבר הקיר, והכיוון האנכי החיובי הוא כלפי מעלה. נניח קרקע אופקית, כבידה קבועה ותנועה ללא התנגדות אוויר.

בנוסחאות הבאות v_0 הוא מהירות השיגור, θ היא זווית השיגור ו־g הוא גודל תאוצת הכובד. חשבו:

- מהירות אופקית: $vx = v_0 \cos(\theta)$
- זמן ההגעה למישור הקיר: $t = wall_x_m / vx$
- גובה מחושב במישור הקיר: $height_at_wall = v_0 \sin(\theta)\, t - g\, t^2 / 2$
- מהירות אנכית במישור הקיר: $vy = v_0 \sin(\theta) - g\, t$

אפשר להשתמש ב־`sind` וב־`cosd`, או להמיר את הזווית לרדיאנים עם `deg2rad`.

הגובה `height_at_wall` מחושב לפי ההמשך המתמטי של המסלול, גם אם הקליע כבר נחת לפני הקיר. גובה שלילי פירושו שהקליע נחת לפני הקיר. במקרה כזה מדפיסים רק הודעה על הנחיתה, ואין צורך לחשב את המהירות האנכית במישור הקיר. אין צורך לדמות התנגשות או לחשב את זמן הנחיתה בנפרד.

הקלט

התוכנית קוראת חמישה מספרים, מספר אחד בכל שורה, בסדר הבא:

סדר	שם המשתנה	משמעות	יחידות
1	`speed_m_s`	מהירות השיגור	$\mathrm{m/s}$
2	`angle_deg`	זווית השיגור	מעלות
3	`wall_x_m`	המרחק האופקי לקיר	m
4	`wall_height_m`	גובה הקיר	m
5	`gravity_m_s2`	גודל תאוצת הכובד	$\mathrm{m/s^2}$

```
Launch speed (m/s):
Launch angle (deg):
Wall distance (m):
Wall height (m):
Gravity (m/s^2):
```

לקריאת כל ערך נשלב שתי פעולות: `readline()` קוראת שורת טקסט, ו־`parse(Float64, ...)` ממירה אותה למספר מסוג `Float64`. פקודות הקלט של הערך הראשון כבר מופיעות בקובץ השלד:

```
println("Launch speed (m/s):")
speed_m_s = parse(Float64, readline())
```

השלימו באותו אופן את פקודות הקלט לארבעת הערכים הנותרים. לכל ערך הדפיסו תחילה את בקשת הקלט המופיעה בטבלה, באמצעות `println`, ואז קראו את המספר. כך כל בקשת קלט תופיע בשורה נפרדת.

לצורך התרגיל אפשר להניח שבכל שורה יוקלד מספר בפורמט ש־`parse` יכולה להמיר. אין צורך לטפל בטקסט כמו `hello`, בשורות חסרות או בבקשת קלט חוזרת לאחר שגיאה.

אילו ערכי קלט מותרים?

כל חמשת הערכים חייבים להיות סופיים. בנוסף:

- `speed_m_s > 0`
- `0 <= angle_deg < 90`
- `wall_x_m >= 0`
- `wall_height_m >= 0`
- `gravity_m_s2 > 0`

לפני החלוקה יש לבדוק גם שהמהירות האופקית חיובית: `vx > 0`.

מכיוון שהקלט מומר באמצעות `parse(Float64, ...)`, כל הערכים שנקראו בהצלחה הם מסוג `Float64`. יש לבדוק שהם סופיים באמצעות `isfinite`, ושהם נמצאים בתחומים המפורטים לעיל. אין צורך בבדיקות סוג נוספות, כגון `isa Real` או בדיקה אם הערך הוא מסוג `Bool`.

אם ערך אינו תקין, יש לעצור את התוכנית באמצעות `throw(ArgumentError("..."))` **לפני חישוב זמן ההגעה, הגובה והמהירות האנכית, ולפני הדפסת שורות התוצאה**. מותר שבקשות הקלט יודפסו לפני הבדיקה. נוסח הודעת השגיאה נתון לבחירתכם.

משתני התוצאה

לכל קלט תקין יש לחשב את `height_at_wall` — הגובה המחושב במיקום הקיר, במטרים.

אם `height_at_wall < 0`, מדפיסים את הודעת הנחיתה שלהלן. במקרה זה אין חובה להגדיר את `vertical_state` ואת `clears`.

אחרת, כאשר `height_at_wall >= 0`, יש לחשב גם את המהירות האנכית `vy` ולשמור את התוצאות במשתנים הבאים:

- `vertical_state` — המחרוזת `"rising"` אם `vy > 0`, ואחרת `"falling"`. לפי מוסכמת התרגיל, גם כאשר `vy == 0` יש להציב `"falling"`.
- `clears` — ערך בוליאני: `true` אם `height_at_wall >= wall_height_m`, ואחרת `false`. משתנה זה מציין אם הגובה המחושב מגיע לפחות לגובה הקיר. לכן גם בפגיעה בדיוק בקצה העליון של הקיר ערכו יהיה `true`, אף שבפלט תופיע הודעה נפרדת למקרה זה.

הפלט

הקליע נוחת לפני הקיר

אם `height_at_wall < 0`, התוכנית מדפיסה **שורת תוצאה אחת בלבד**:

```
Result: The projectile lands before the wall.
```

במקרה זה אין להדפיס את השורות `Height at wall` ו־`Vertical motion`.

הקליע מגיע למיקום הקיר בגובה שאינו שלילי

אם `height_at_wall >= 0`, התוכנית מדפיסה **בדיוק שלוש שורות תוצאה**:

```
Height at wall: <number> m
Vertical motion: <state>
Result: <result>
```

בשורה הראשונה מציגים את הגובה שחישבתם. מותר להשתמש בהצגה המספרית הרגילה של Julia; בבדיקה תתקבל סטייה של עד `0.0001` מטר מהערך הצפוי. אפשר גם לעגל את הערך המוצג לארבע ספרות אחרי הנקודה, אך את התנאים יש לבדוק באמצעות הגובה המקורי, לפני העיגול.

בשורה השנייה `<state>` הוא בדיוק `rising` או `falling`.

השורה השלישית חייבת להתאים בדיוק לאחד משלושת הנוסחים הבאים:

תנאי	שורת הפלט השלישית
`0 <= height_at_wall < wall_height_m`	`Result: The projectile hits the wall.`
`height_at_wall == wall_height_m`	`Result: The projectile hits the edge of the wall.`
`height_at_wall > wall_height_m`	`Result: The projectile clears the wall.`

השתמשו בתנאי חיצוני כדי להבחין בין נחיתה לפני הקיר לבין הגעה למיקום הקיר בגובה שאינו שלילי. בענף השני השתמשו בתנאים מקוננים כדי לקבוע את המצב האנכי ואת תוצאת המפגש עם הקיר. כך שורות הגובה והמצב האנכי יודפסו רק בענף המתאים.

מלבד בקשות הקלט ושורות התוצאה המתאימות למקרה, אין להדפיס כותרות, הודעות נוספות או ערכי ביניים.

לבדיקת פגיעה בקצה העליון של הקיר השתמשו ב־`==` על הערכים לפני העיגול. מקרה השוויון שבטבלת הבדיקה העצמית משתמש במרחק קיר אפס ובגובה קיר אפס, ולכן מתקבל בו שוויון מדויק. בתרגיל זה אין צורך להוסיף סבולת לבדיקת השוויון.

עבור `wall_height_m = 7.0  ,wall_x_m = 10.0  ,angle_deg = 45.0  ,speed_m_s = 20.0` ו־`gravity_m_s2 = 9.81`, שלוש שורות התוצאה הן:

```
Height at wall: 7.5475 m
Vertical motion: rising
Result: The projectile clears the wall.
```

בדיקה עצמית

בדקו את התוכנית בעזרת ששת המקרים הבאים. בכל המקרים `gravity_m_s2 = 9.81`. עמודות הגובה והמצב האנכי מציינות את הפלט הצפוי; הגובה נתון במטרים. הסימן „—" מציין ששורה זו אינה מודפסת. תיאורי התוצאה המקוצרים מפנים להודעות המלאות שבסעיף הפלט.

speed_m_s	angle_deg	wall_x_m	wall_height_m	Height (m)	State	Result
20	45	10	7	7.5475	rising	clears
20	45	10	8	7.5475	rising	hits wall
20	45	30	7	7.9275	falling	clears
20	45	30	8	7.9275	falling	hits wall
20	45	50	2	—	—	lands before wall
20	0	0	0	0.0	falling	hits edge

במקרה הנחיתה לפני הקיר, הגובה המחושב הוא בקירוב `11.3125` – מטר. משתמשים בו לבדיקת התנאי, אך אין להדפיס אותו. הפלט במקרה זה הוא שורת הודעת הנחיתה בלבד.

המקרה האחרון בודק שוויון מדויק בגובה. הוא מלאכותי מבחינה פיזיקלית, אך שימושי כבדיקת גבול.

בדקו גם קלטים שאינם תקינים: מהירות אפס או שלילית, זווית שלילית או זווית של `90` מעלות ומעלה, מרחק קיר שלילי, גובה קיר שלילי, תאוצת כובד אפס או שלילית, וערך שאינו סופי, כגון `Inf` או `NaN`. בכל הרצה כזאת התוכנית צריכה לעצור עם שגיאת `ArgumentError`, בלי להדפיס שורות תוצאה.

לצורך הבדיקה יוכל הסגל להריץ את התוכנית ולספק לה חמישה מספרים באותו סדר שבו אתם מקלידים אותם. בבדיקת הפלט מתעלמים מבקשות הקלט ובודקים שורת תוצאה אחת במקרה של נחיתה לפני הקיר, ושלוש שורות בשאר המקרים. כאשר הגובה מודפס, משווים אותו בקירוב; את הודעות המצב והתוצאה משווים בדיוק. בנוסף, יש לוודא בקוד שהמשתנה `clears` מחושב לפי ההגדרה שלעיל בענף שבו הגובה אינו שלילי. בדיקות נוספות ישתמשו רק בדרישות ובתחומי הקלט המפורטים בדף זה.

עבודה והגשה

1. פתחו את `HW2_student_template.jl` ב־VS Code ושמרו עותק בשם `YOUR_ID.jl`.
2. השלימו את פקודות הקלט לארבעת הערכים החסרים, ואז כתבו את החישובים, התנאים ופקודות הפלט בין `=== BEGIN STUDENT CODE === #` ובין `# === END STUDENT CODE ===`. החליפו את שורת `error(...)` בפתרון שלכם, והשאירו את שורות הסימון ללא שינוי.
3. פתחו מסוף בתיקייה שבה שמרתם את הקובץ, הריצו `julia YOUR_ID.jl` והקלידו מספר אחד בכל שורה. החליפו את `YOUR_ID` במספר הסטודנט שלכם, ובדקו את המקרים המופיעים לעיל.
4. הגישו את **הקובץ השלם** בעמוד ה־Moodle של הקורס.

אין צורך להתקין חבילות נוספות, לקרוא קבצים מתוך התוכנית, להגדיר פונקציות או לכתוב לולאות.